

Topic-1: Introduction to Interfaces

- The interface in Java is a mechanism to achieve abstraction. There can be only abstract methods in the Java interface, not method body.
- Since Java 8, we can have **default and static methods** in an interface.
- Since Java 9, we can have **private methods** in an interface.

There are mainly three reasons to use interface. They are given below.

- It is used to achieve abstraction.
- By interface, we can support the functionality of multiple inheritance.
- It can be used to achieve loose coupling.

Declaring an interface:

- An interface is declared by using the interface keyword.
- It provides total abstraction; means all the methods in an interface are declared with the empty body, and all the fields are public, static and final by default.
- A class that implements an interface must implement all the methods declared in the interface.

Syntax:

```
interface <interface_name>{  
    // declare constant fields  
    // declare methods that are abstract  
    // by default.  
}
```

Important Points about Interface in Java:

1. `interface` is the keyword that is used to create an interface in java.
2. We can't instantiate an interface in java i.e., we can't create object for an interface in java
3. Interfaces can't have constructors because we can't instantiate them and interfaces can't have a method with body.
4. By default any **attribute** of interface is **public**, **static** and **final**, so we don't need to provide access modifiers to the attributes but if we do, compiler doesn't complain about it either.
5. By default interface **methods** are implicitly **abstract** and **public**.
6. An interface can't extend any class but it can extend another interface.
7. Java provides multiple inheritance in interfaces, what it means is that an interface can extend multiple interfaces.
8. `implements` keyword is used by classes to implement an interface.
9. A class implementing an interface must provide implementation for all of its method unless it's an abstract class.

Example:

```
interface Shape {

    //implicitly public, static and final
    String LABEL="Shape";

    //interface methods are implicitly abstract and public
    void draw();

    double getArea();

}

class Circle implements Shape {

    private double radius;

    Circle(double r){
        this.radius = r;
    }

    @Override
    void draw() {
        System.out.println("Drawing Circle");
    }

    @Override
    double getArea(){
        return Math.PI*this.radius*this.radius;
    }

    double getRadius(){
        return this.radius;
    }

}

class ShapeTest {

    public static void main(String[] args) {

        //programming for interfaces not implementation
        Shape shape = new Circle(10);

        shape.draw();
        System.out.println("Radius="+shape.getRadius());
        System.out.println("Area="+shape.getArea());

    }

}
```

Output:

```
Drawing Circle
Radius: 10.000000000000000
Area=314.1592653589793
```

Topic-2: default methods and static methods in interfaces

- The Java 8 release introduces or allows us to have **static** and **default** methods in the interfaces.
- Using default methods in an interface, the developers can add more methods to the interfaces. This way they do not disturb or change the classes that implement the interface.
- Java 8 also allows the interface to have a static method. Static methods are the same as those we define in classes. Note that the static method cannot be overridden by the class that implements the interface.

Example for default method in an interface:

```
interface TestInterface {
    // abstract method
    public void cubeNumber(int num);

    // default method
    default void print()
    {
        System.out.println("TestInterface :: Default method");
    }
}

class TestClass implements TestInterface {
    // override cubeNumber method
    public void cubeNumber(int num)
    {
        System.out.println("Cube of given number " + num+ ":" +
num*num*num);
    }
}

class Main{
    public static void main(String args[])
    {
        TestClass obj = new TestClass();
        obj.cubeNumber(5);

        // call default method print using class object
        obj.print();
    }
}
```

Output:

```
Cube of given number 5:125
TestInterface :: Default method
```

Example for static method in an interface:

```
//interface declaration
interface TestInterface {
    // static method definition
    static void static_print() {
        System.out.println("TestInterface::static_print  ( )");
    }
    // abstract method declaration
    void nonStaticMethod(String str);
}

// Interface implementation
class TestClass implements TestInterface {
    // Override interface method
    @Override
    public void nonStaticMethod(String str) {
        System.out.println(str);
    }
}

public class Main{
    public static void main(String[] args) {
        TestClass classDemo = new TestClass();

        // Call static method from interface
        TestInterface.static_print();

        // Call overridden method using class object
        classDemo.nonStaticMethod("TestClass::nonStaticMethod ( )");
    }
}
```

Output:

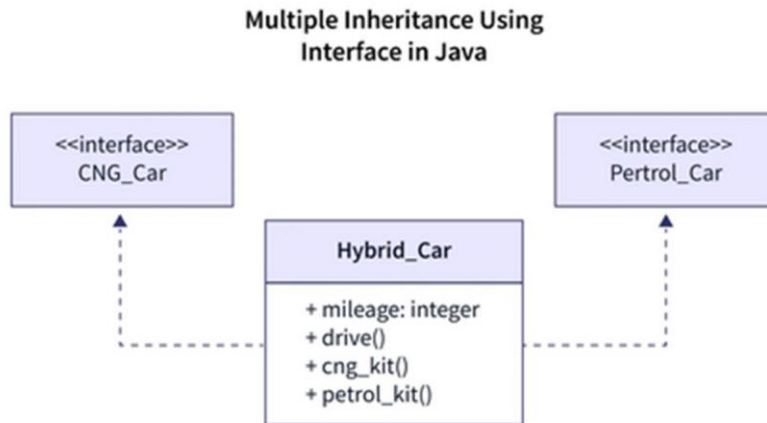
```
TestInterface::static_print ( )
TestClass::nonStaticMethod ( )
```

Topic-3: Multiple inheritance through interface

An **interface** is a fully abstract class i.e., it includes a set of abstract methods that specifies what a class must do and not how to do it. Hence, if a class implements an interface, it inherits all the abstract methods of the interface and provides an implementation for each of them.

Interfaces are non-instantiable and provide a representation of what a class should implement i.e., interfaces do not contain detailed instructions for their behaviors. Because of this feature, a class can implement any number of interfaces without any ambiguity as the implementation is provided by the class itself.

Now, we can effectively bypass the diamond problem and implement multiple inheritance in Java using the interface

**Program:**

```
// CNG Car interface
interface CNG_Car{
    // Abstract methods
    void drive();
    void cng_kit();
}

// Petrol Car interface
interface Petrol_Car{
    // Abstract methods
    void drive();
    void petrol_kit();
}

// Multiple Inheritance using Interface
class Hybrid_Car implements Petrol_Car, CNG_Car {
    public void drive(){
        System.out.println("Driving a Hybrid Car");
    }

    // Overridden method of CNG_Car Interface
    public void cng_kit(){
        System.out.println("Using the CNG kit for Hybrid Car");
    }

    // Overridden method of Petrol_Car Interface
    public void petrol_kit(){
        System.out.println("Using the Petrol kit for Hybrid Car");
    }
}

// Driver Code
class Main {
    public static void main(String args[]) {
        // Creating a new object of the Hybrid Car class
        Hybrid_Car obj = new Hybrid_Car();
    }
}
```

```
// Calling the methods of the Hybrid_Car class
obj.drive();
obj.cng_kit();
obj.petrol_kit();
    }
}
```

Output:

Driving a Hybrid Car

Using the CNG kit for Hybrid Car

Using the Petrol kit for Hybrid Car

Topc-4: Nested interface

- In java, an interface may be defined inside another interface, and also inside a class.
- The interface that defined inside another interface or a class is known as nested interface.
- The nested interface is also referred as inner interface.

4.1 Nested interface inside another interface

The nested interface that defined inside another interface must be accessed as OuterInterface.InnerInterface.

Example:

```
interface OuterInterface{
    void outerMethod();

    interface InnerInterface{
        void innerMethod();
    }
}

class OnlyOuter implements OuterInterface{
    public void outerMethod() {
        System.out.println("This is OuterInterface method");
    }
}

class OnlyInner implements OuterInterface.InnerInterface{
    public void innerMethod() {
        System.out.println("This is InnerInterface method");
    }
}

public class NestedInterfaceExample {

    public static void main(String[] args) {
        OnlyOuter obj_1 = new OnlyOuter();
        OnlyInner obj_2 = new OnlyInner();
    }
}
```

```
        obj_1.outerMethod();
        obj_2.innerMethod();
    }
}
```

Output:

```
This is OuterInterface method
This is InnerInterface method
```

4.2 Nested interface inside a class

The nested interface that defined inside a class must be accessed as `ClassName.InnerInterface`.

Example:

```
class OuterClass{
    interface InnerInterface{
        void innerMethod();
    }
}

class ImplementingClass implements OuterClass.InnerInterface{
    public void innerMethod() {
        System.out.println("This is InnerInterface method");
    }
}

public class NestedInterfaceExample {
    public static void main(String[] args) {
        ImplementingClass obj = new ImplementingClass();

        obj.innerMethod();
    }
}
```

Output:

```
This is InnerInterface method
```

Topc-5: Extending an interface

- In java, an interface can extend another interface.
- When an interface wants to extend another interface, it uses the keyword **extends**.
- The interface that extends another interface has its own members and all the members defined in its parent interface too.
- The class which implements a child interface needs to provide code for the methods defined in both child and parent interfaces, otherwise, it needs to be defined as abstract class.

Example

```
interface ParentInterface{
    void parentMethod();
}

interface ChildInterface extends ParentInterface{
    void childMethod();
}

class ImplementingClass implements ChildInterface{

    public void childMethod() {
        System.out.println("Child Interface method!!");
    }

    public void parentMethod() {
        System.out.println("Parent Interface method!");
    }
}

public class ExtendingAnInterface {

    public static void main(String[] args) {

        ImplementingClass obj = new ImplementingClass();

        obj.childMethod();
        obj.parentMethod();

    }
}
```

Output:

```
Child Interface method!!
Parent Interface method!
```

Topic-6: Annotations in Java

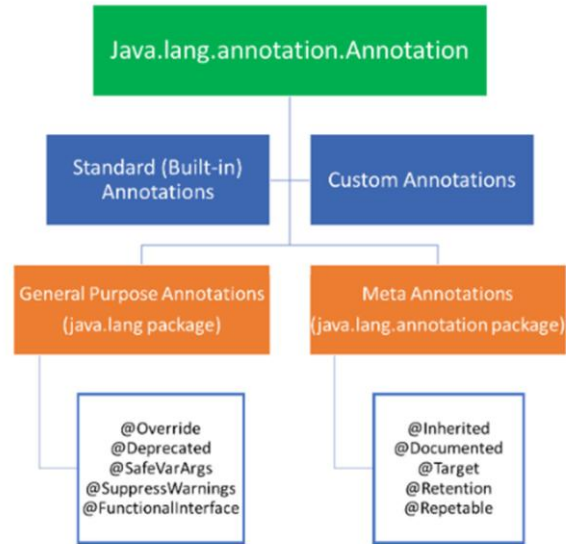
Annotations in Java provide additional information to the compiler and JVM. An annotation is a tag representing metadata about classes, interfaces, variables, methods, or fields.

Annotations do not impact the execution of the code that they annotate.

Some of the characteristics of annotations are:

- Begin with '@'
- Do not alter the execution of the program
- Provide supplemental information and help to link metadata with elements of a program such as classes, variables, constructs, methods, etc.
- Are different from comments since they can affect how the program is treated by the compiler

Hierarchy of Annotations in Java



Example 1: @Override Annotation Example

```
class Animal {
    public void displayInfo() {
        System.out.println("I am an animal.");
    }
}

class Dog extends Animal {
    @Override
    public void displayInfo() {
        System.out.println("I am a dog.");
    }
}

class Main {
    public static void main(String[] args) {
        Dog d1 = new Dog();
        d1.displayInfo();
    }
}
```

Output :

I am a dog.

Topic-7: Functional Interfaces

A functional interface is an interface that has only one abstract method. It can contain any number of default and static methods but the abstract method it contains is exactly one. Additionally, a functional interface can have declarations of object class methods.

Functional Interface is known as “**Single Abstract Method Interface**” or “**SAM Interface**”. SAM interface is a new feature in Java.

In a Java program, the presence of a functional interface is indicated by using a **@FunctionalInterface** annotation. When the compiler encounters this annotation, then it knows that the interface that is following this annotation is functional. Hence if it contains more than one abstract method, then it flashes an error.

Example:

```
/declare a functional interface
@FunctionalInterface    //annotation indicates it's a functional interface
interface function_Interface{
    void disp_msg(String msg);    // abstract method
}
//implementation of Functional Interface
class FunctionalInterfaceExample implements function_Interface{
    public void disp_msg(String msg){
        System.out.println(msg);
    }
}
class Main{
    public static void main(String[] args) {
        //create object of implementation class and call method
        FunctionalInterfaceExample finte = new
FunctionalInterfaceExample();
        finte.disp_msg("Hello, World!!!");
    }
}
```

Output:

```
Hello, World!!!
```